NOIP 2026 模拟赛 题解

HEZ-MPLN&GdxMilk

题目名称	无限街	必经路	排列题	君往何处
题目类型	传统型	传统型	传统型	传统型
目录	`street`	`path`	`perm`	`quovadis`
可执行文件名	`street`	`path`	`perm`	`quovadis`
输入文件名	`street.in`	`path.in`	`perm.in`	`quovadis.in`
输出文件名	`street.out`	`path.out`	`perm.out`	`quovadis.out`
每个测试点时限	1.0 秒	1.0 秒	0.5 秒	2.5 秒
内存限制	512 MiB	512 MiB	128 MiB	512 MiB
测试点数目	6	8	20	10
测试点是否等分	否	否	是	否

提交源程序文件名

对于 C++ 语言	`street.cpp`	`path.cpp`	`perm.cpp`	`quovadis.cpp`

编译选项

对于 C++ 语言	`-O2 -std=c++14 -static`

无限街（street）

by MPLN。

签到题。

观察到两个相邻的车离开时间一定相差 2。所以对于每个车串，求出第一辆车离开时间就可以等差数列求和算出全部离开时间的和。

对于一辆车，其前面所有车串肯定最终都会被“拉长”到每辆车位置相差 2。

假设前面所有车串长度之和为 l，则会多出 l 的“缓冲空位”（每辆车后面一个“缓冲空位”）。

但是初始车串之间还有空隙，可以用空隙长度的时间抵消部分“缓冲空位”耽误的时间。

按顺序遍历车串，维护一个延迟时间 c，遇到一个长度为 l 的空隙就 $c \leftarrow \max(0, c-l)$，遇到长度为 l 的车串就 $c \leftarrow c+l$。然后遍历到一个新车串 i 时，其开头车辆的离开时间就是 $l_i + c$，其他车辆可以推算。

时间复杂度 $O(n)$。

必经路（path）

sol by MPLN，原题 LG12584。

设额外边集为 G，对每个点 i 与 $i+1$ 之间，定义 $E_i=\{(u,v)\in G\mid u\le i<v\}$。

情况一：桥

当 $|E_i|=0$，路径边 $(i,i+1)$ 是桥，损坏桥，对答案贡献 $2i(n-i)$，若损坏两条桥，它们的贡献直接相加，所以取贡献最大的两条桥。

情况二：前缀两边割

若路径边 $(i,i+1)$ 满足 $|E_i|=1$，则损坏这两条贡献为 $2i(n-i)$。

情况三：内部区间两边割

考虑区间 $[p+1,q]$，其路径边界是 $(p,p+1)$ 和 $(q,q+1)$，没有额外边从区间内部连到外部，当且仅当 $E_p=E_q$。

删除两条路径边后正好分成两个连通块，大小分别为 $q-p,n-(q-p)$，贡献 $2(q-p)(n-q+p)$，扫描所有 $(i,i+1)$，给额外边赋随机值，用 64 位异或哈希表示 E_i，将相同集合的分组。组内对每个元素只需寻找距离最接近 $\frac{n}{2}$ 的一对位置（不难证明取最大值）。

哈希等操作均可用差分简单解决。

复杂度 $O(m+n\log n)$。

排列题（perm）

sol by GdxMilk，原题 CF1988F。

题解 from int4399。

首先答案要求一个排列的信息，要求前缀最大值和前缀最小值，我们考虑从大到小插入。又发现插入 n 之后右边没有前缀最大值，左边没有后缀最大值，可以分开做，最后合并。

这里只写前缀方案的转移，后缀同理。

设 $f_{i,j,k}$ 表示插了 i 个数，有 j 个前缀最大值，有 k 个上升点的方案。

- i 插前面：$f_{i,j,k} \leftarrow f_{i-1,j-1,k-1}$
- i 插最后或一个上升点后面：$f_{i,j,k} \leftarrow (k+1)f_{i-1,j,k}$
- 其他：$f_{i,j,k} \leftarrow (i-k-1)f_{i-1,j,k-1}$

那么答案就是枚举 n 再枚举两边放数的方案，有

$$ans_n = \sum_{p=1}^{n} \binom{n-1}{p-1} \times \left(\sum_{i=0}^{p-1} \sum_{j=0}^{n-p} \sum_{x=0}^{p-1} \sum_{y=0}^{n-p} f_{p-1,i,x} \times g_{n-p,j,y} \times a_{i+1} \times b_{j+1} \times c_{x+y+[p>1]} \right)$$

主要是里面的式子，交换下求和顺序再拆开，可以得到原式等于

$$\sum_{x=0}^{p-1} \sum_{y=0}^{n-p} c_{x+y+[p>1]} \times s_{p-1,x} \times t_{n-p,y}$$

其中

$$s_{i,j} = \sum_{x=0}^{i} f_{i,x,j} \times a_{x+1}$$

$$t_{i,j} = \sum_{y=0}^{i} g_{i,y,j} \times b_{j+1}$$

还得再拆一层。发现 s 和 y 无关，考虑把

$$c_{x+y+[p>1]} \times t_{n-p,y}$$

再压，设

$$S_{i,j} = \sum_{a=x}^{i} c_{a+j} \times t_{i,a}$$

有

$$ans_i = \sum_{p=1}^{i} \binom{i-1}{p-1} \times \sum_{x=0}^{p-1} \left(s_{p-1,x} \times S_{i-p,x+[p>1]} \right)$$

现在可以直接做了，式子自己推。

君往何处（quovadis）

sol by MPLN，原题 CF1776E。

闲话：题目答案不多，看着就容易乱搞，和 AI 大力合作造的数据~~不一定保证强度~~。

题解 from UltiMadow。

考虑画出人的路程 - 时间图像，不难发现题目要求的是一条不碰到任意一条给定的横线的折线。

由于人可以在任意时刻改变速度，那么要求还可以转化为折线不穿过任意一条给定的横线，但可以沿着横线走（因为对于和横线重合的部分，我们可以沿着横线一端旋转一个很小的角度）。

此时不难发现折线的每一段对应的直线只有 $O(n^2)$ 条，即只可能是某两条横线的端点构成的直线，于是就可以 dp 了。

为了方便，我们添加一条 $(0,0)-(0,s)$ 的横线和一条 $(m+1,0)-(m+1,s)$ 的横线，标号分别为 0 和 $n+1$。

先对于每一条直线预处理出 ql_i 表示这条直线向下最多延伸到的位置，qr_i 表示这条直线向上最多延伸到的位置。

记 $f_{t,i}$ 为走到第 i 条直线纵坐标为 t 的位置最少需要改变多少次速度，dn_i 为走到第 i 条横线下方最少需要改变多少次速度，up_i 为走到第 i 条横线上方最少需要改变多少次速度。

从小到大枚举 t，考虑如何转移。

先考虑直线与横线之间的转移，不妨先将直线按纵坐标为 t 时横坐标大小从小到大排序，横线按左端点大小排序，并把有交的横线并起来。

枚举横线 i，记其左端点为 l_i，右端点为 r_i，再枚举直线 j，记其在纵坐标为 t 时的横坐标为 $pos_{t,j}$，那么有转移：

1. 若 $pos_{t,j}\le l_i$，则 $up_i\to f_{t,j}+1$；
2. 若 $pos_{t,j}\ge l_i$，则 $f_{t,j}\to up_i+1$；
3. 若 $pos_{t,j}\le r_i$，则 $dn_i\to f_{t,j}+1$；
4. 若 $pos_{t,j}\ge r_i$，则 $f_{t,j}\to dn_i+1$。

再考虑直线与直线之间的转移，$f_{t,i}$ 能转移到 $f_{t,j}$ 当且仅当直线 i 与直线 j 在纵坐标 $[t,t+1]$ 范围内有交，也即在按照 pos_t 排序的情况下 pos_{t+1} 构成逆序对，于是可以直接用树状数组优化转移。

注意要特殊处理 $pos_{t,i}=pos_{t,j}$ 和 $pos_{t+1,i}=pos_{t+1,j}$ 的情况。

时间复杂度 $O(n^3+n^2m\log n)$。

直线之类的操作应该可以直接拿浮点数做，不过也可以乘上 $\gcd(1,2,...,11)=55440$ 规避掉浮点数。